

Concurrency Issues and Tradeoffs: Elixir and Haskell

Generated: April 24, 2026

This brief highlights concurrency-related risks when combining Elixir and Haskell in one platform, and defines a practical boundary model for a solo-developer refactor.

Executive Summary

- There is no fundamental language mismatch between Elixir and Haskell in your architecture.
- The main risks are boundary and ownership risks, not runtime capability gaps.
- Elixir is strongest as orchestration and fault-management control plane.
- Haskell is strongest as deterministic compute plane for rules and evaluation.

How Their Concurrency Models Differ

- Elixir uses actor-style processes with isolated state and message passing.
- Haskell uses lightweight threads plus abstractions such as STM, MVar, and Async.
- Elixir encourages supervision-first design through OTP behaviors.
- Haskell encourages explicit composition of concurrency patterns and effects.

Primary Integration Risks

- Reliability semantics mismatch: fire-and-forget delivery between services can lose work.
- State ownership mismatch: copied schedule/config state across two databases can drift.
- Contract mismatch: idempotency, retries, and timeout behavior are not strict enough.
- Observability mismatch: weak end-to-end correlation IDs make incidents harder to debug.

What to Keep in Elixir

- Scheduling and cron ownership (Quantum jobs, source polling, retries).
- Operational workflow state (job runs, dead-letter outcomes, audit trail).
- Service boundary hardening (auth, request signing, backoff policies).
- User-facing operations (batch visibility, status, controls).

What to Keep in Haskell

- Rule parsing and deterministic claim evaluation.
- Pure, testable decision logic independent of transport/scheduler concerns.
- Parallel compute internals where useful (Async/STM) without owning orchestration state.
- Stable request-response evaluation API with strict input/output schema.

Guardrails to Prevent Concurrency Failures

- Define idempotency keys for each batch and claim-level ingest action.
- Use explicit retry classes: retryable vs non-retryable errors.
- Set timeout budgets per hop and enforce cancellation propagation.
- Propagate `request_id` and `batch_id` through all logs and events.
- Persist delivery attempts and outcomes; do not rely on transient logs.

Demo-Stage Baseline (Not Full Production Hardening)

- Authenticated UI with role separation for upload/schedule/result access.
- Signed service-to-service requests between orchestrator and translator.
- Basic retry with bounded backoff and visible failed-run queue.
- Simple run-history dashboard and per-batch traceability.

Recommended Next Steps

- Consolidate scheduler ownership into Elixir claims checker.
- Reduce translator responsibilities to stateless translation service.
- Freeze and version the cross-service JSON contract before refactor.
- Run parallel validation for 1-2 weeks before full cutover.